

Vectorización por lotes del ensamblaje

Propuesta de diseño para la deuda técnica #19 de Solidum FEM

Documento de trabajo para el ADR correspondiente

22 de septiembre de 2026

Resumen

El ensamblaje de Solidum FEM recorre los elementos uno a uno en Python y, dentro de cada uno, los puntos de Gauss uno a uno. Un perfil sobre una malla de 10 000 Quad4 muestra que más del 90 % del tiempo es sobrecarga de despacho (llamadas a funciones, creación de arreglos pequeños, diccionarios de estado) y no aritmética. Este documento propone reorganizar el ensamblaje “por lotes”: agrupar los elementos por familia derivada de sus contratos declarativos, evaluar la cinemática y la constitutiva de todos los puntos de Gauss de la familia dentro de un único kernel compilado, y almacenar el estado interno como arreglos en vez de diccionarios. La ganancia esperada es de 30 a 100 veces en el ensamblaje y de un orden de magnitud en la memoria del estado interno, sin cachear la matriz $\mathbf{B}$ (que duplicaría la memoria por elemento) y sin hipótesis sobre la cinemática (el kernel recomputa siempre desde coordenadas de referencia y desplazamientos). El diseño se ciñe al principio del proyecto de no particularizar al presente: la agrupación se deriva de los contratos, el camino por elemento sigue siendo el contrato obligatorio y el camino por lotes es una capacidad que cada componente declara, con equivalencia bit a bit exigida en el barrido de registros. No se implementa nada hasta que se apruebe el ADR.

Índice

1. Motivación: dónde se va el tiempo y la memoria	2
1.1. Perfil del ensamblaje actual	2
1.2. Memoria del estado interno	2
2. Diagnóstico: sobrecarga de despacho, no aritmética	2
3. Principios de diseño	3
4. Arquitectura propuesta	3
4.1. Familias de lote	4
4.2. Contrato opcional del elemento	4
4.3. Contrato opcional del material	5
4.4. Estado interno por arreglos	5
4.5. Cinemática	5
4.6. Ensamblador	6
5. Memoria	6
5.1. Lo que se ahorra	6
5.2. Lo que no se debe cachear	6

5.3. Temporales acotados por trozos	7
5.4. El techo real	7
6. Rendimiento esperado y cómo validarlo	7
7. Verificación	7
8. Plan por fases	8
8.1. Coste del componente $N + 1$	9
9. Alternativas consideradas y descartadas	9
10. Decisiones abiertas para el ADR	9
A. Reproducción de las mediciones	10

1 Motivación: dónde se va el tiempo y la memoria

1.1 Perfil del ensamblaje actual

La tabla 1 recoge el perfil (`cProfile`) de un ensamblaje no lineal de una malla regular de 100×100 Quad4 con `VonMises2D` en rama elástica, es decir, 10 000 elementos y 40 000 puntos de Gauss. El tiempo total con el perfilador activo es 1,97 s; sin perfilador, 1,66 s.

Función	Tiempo propio	Llamadas	Coste unitario
<code>Quad4.compute_element_state</code> (bucle de Gauss y despacho a kernels)	0,96 s	10 000	96 μ s por elemento
<code>VonMises2D.compute_state</code> (envoltorio Python del kernel)	0,38 s	40 000	9 μ s por punto
<code>Assembler.assemble_non_linear_system</code> (bucle, slicing, volcado)	0,16 s	1	16 μ s por elemento
<code>Element.get_coordinate_matrix</code> (bucle Python sobre nodos)	0,13 s	10 000	13 μ s por elemento
<code>Assembler.commit_all_states</code> (deepcopy de 40 000 dicts)	0,40 s	1	10 μ s por punto

Tabla 1: Perfil de un ensamblaje de 10 000 Quad4 con J2 (Python 3.14, Windows).

Un análisis de Newton con 5 pasos y 4 iteraciones por paso son 25 ensamblajes y 5 commits: unos 45 s para 10 000 elementos y del orden de 8 minutos para 100 000. Tras la auditoría de 2026-09-22 cada iteración de Newton ensambla una sola vez (antes dos), lo que ya redujo a la mitad el coste de todo análisis no lineal sin tocar contratos; el resto de este documento trata lo que queda.

1.2 Memoria del estado interno

La tabla 2 muestra lo que ocupa el estado interno en la misma malla, medido con `tracemalloc`. Cada punto de Gauss guarda un diccionario Python con dos arreglos NumPy pequeños (`eps_p` de cuatro componentes y `alpha` escalar). Las mismas magnitudes almacenadas como dos arreglos por familia ocupan once veces menos, y el commit deja de copiar.

Concepto	Memoria	Por punto de Gauss
Estado trial como diccionarios, tras un ensamblaje	17,6 MB	462 B
Estado committed (copia profunda en el commit)	17,6 MB	462 B
Mismo estado como arreglos, trial y committed juntos	3,05 MB	80 B
Matriz K dispersa en CSR	4,2 MB	—
Topología COO cacheada (filas, columnas y datos)	9,8 MB	—

Tabla 2: Memoria del estado interno frente a la matriz global (10 000 Quad4, J2).

Hoy el estado interno es la partida de memoria más grande del ensamblaje, mayor que la matriz global, y se duplica durante el análisis por la copia profunda.

2 Diagnóstico: sobrecarga de despacho, no aritmética

El patrón actual es “un elemento por vez”: cada elemento recorre sus puntos de Gauss en Python y en cada uno realiza una decena de operaciones sobre matrices de 3×8 u 8×8 . NumPy y Numba son rápidos por operación grande, pero cada llamada tiene un coste fijo τ de entre uno y tres

microsegundos, sea sobre una matriz de 8×8 o de 8000×8000 . Con n_{gp} puntos de Gauss y n_{ops} llamadas por punto, el tiempo por elemento es

$$t_{elem} \approx n_{gp} n_{ops} \tau + t_{arit}, \quad (1)$$

con t_{arit} la aritmética real. Para un Quad4 con J2, $n_{gp} = 4$, $n_{ops} \approx 15$ a 20 y $\tau \approx 1,5 \mu s$ dan entre 90 y $120 \mu s$, mientras que t_{arit} (unos cientos de operaciones de coma flotante por punto) vale menos de un microsegundo. La relación es del orden de 100 a 1 .

Los kernels @njit existentes (`_compute_kinematics`, `_compute_integrands`, los return mapping de J2 y Drucker-Prager) aceleran lo que ocurre dentro de ellos, pero no el hecho de invocarlos $80\,000$ veces desde Python: la ley de Amdahl limita la ganancia de compilar el interior de un bucle cuya sobrecarga está en el exterior. A esto se suman tres costes de la misma naturaleza:

- el diccionario de estado por punto de Gauss (creación, `dict.get`, asignación en `vars_trial`);
- `copy.deepcopy` en cada commit, que recorre $40\,000$ diccionarios;
- `get_coordinate_matrix`, un doble bucle Python que reconstruye en cada evaluación las coordenadas de un elemento cuya geometría de referencia no cambia.

Conclusión del diagnóstico

El problema no es la aritmética ni el álgebra dispersa (la conversión COO a CSR cuesta 10 ms). Es la granularidad: hay que invertir el bucle para que la operación sea grande y el despacho, único.

3 Principios de diseño

Estos cinco principios recogen lo acordado con el usuario el 2026-09-22 y el criterio de `Reglas.md` §1: toda decisión se evalúa contra el coste de añadir el componente $N + 1$.

1. **Agrupación derivada de los contratos, no de listas.** La clave del lote se construye con los atributos declarativos que ya existen (clase de elemento, clase de material, regla de cuadratura, número de nodos). Un elemento nuevo cae en su propio lote sin tocar el ensamblador.
2. **El camino por elemento sigue siendo el contrato obligatorio.** `compute_element_state` y `compute_state` no cambian. El camino por lotes es una capacidad que el componente declara; si no la declara, el ensamblador lo procesa por el camino de siempre. Un material de investigación se implementa primero de forma simple y se acelera después si hace falta.
3. **El estado interno se declara, no se cablea.** Cada material declara el esquema de sus variables (nombres y formas) y el almacenamiento por arreglos se genera de esa declaración. Un estado que no encaje en arreglos queda automáticamente en el camino por elemento.
4. **El kernel no supone nada sobre la cinemática.** Recibe siempre coordenadas de referencia y desplazamientos y recomputa lo que necesite. Nada derivado de $\mathbf{x} = \mathbf{X} + \mathbf{u}$ se cachea entre iteraciones; vale para elementos lineales, corotacionales y para futuras formulaciones lagrangianas total o actualizada.
5. **Equivalencia demostrada, no supuesta.** El camino por elemento es la referencia física; el camino por lotes debe reproducirlo a precisión de máquina en toda combinación registrada, y la comprobación vive en el barrido de contratos que ya recorre los tres registros.

4 Arquitectura propuesta

4.1 Familias de lote

Una *familia* es el conjunto de elementos del dominio que comparten clase de elemento, clase de material, regla de cuadratura y número de nodos. La clave se deriva de atributos que ya existen:

```
def family_key(elem):
    return (type(elem), type(elem.material), elem.quadrature_key
            if elem.quadrature_key is not None else id(elem.points),
            len(elem.nodes))
```

Los parámetros que varían de un elemento a otro dentro de la familia (espesor en 2D; área e inercias en 1D) no forman parte de la clave: el elemento los declara en `BATCH_PARAMS` y el constructor de la familia los reúne en un arreglo (N, n_{par}) . Lo mismo ocurre con los parámetros del material, que son constantes por familia porque la instancia del material es la misma para todos sus elementos.

El `Assembler` construye las familias una sola vez, junto con la topología COO que ya cachea, y las invalida con la misma huella de topología introducida en la auditoría. Para cada familia guarda los índices globales de sus elementos (N, n_{dof}) y el desplazamiento `ptr` de cada uno dentro del vector `data` de la COO.

4.2 Contrato opcional del elemento

El elemento que quiera participar en el camino por lotes declara dos atributos de clase:

```
class Quad4(Element):
    BATCH_PARAMS = ("thickness",)          # escalares por elemento
    BATCH_KERNEL = _quad4_batch_kernel    # funcion @njit, None por defecto
```

El kernel recibe únicamente arreglos y un kernel puntual del material, y devuelve arreglos:

```
@njit(cache=True)
def _quad4_batch_kernel(X, u, params, gp_pts, gp_w,
                        mat_kernel, mat_params, S_in, S_out,
                        K_out, F_out, eps_out, sig_out):
    # X: (N, 4, 2) coordenadas de referencia; u: (N, 8) desplazamientos
    # S_in/S_out: (N*n_gp, n_state) estado committed / trial
    n_gp = gp_pts.shape[0]
    for e in range(X.shape[0]):
        K = np.zeros((8, 8)); F = np.zeros(8)
        for g in range(n_gp):
            B, detJ = _compute_kinematics(gp_pts[g, 0], gp_pts[g, 1], X[e])
            strain = B @ u[e]
            row = e * n_gp + g
            sigma, C, S_out[row] = mat_kernel(strain, S_in[row], mat_params)
            dV = detJ * gp_w[g] * params[e, 0]
            K += (B.T @ C @ B) * dV
            F += (B.T @ sigma) * dV
            eps_out[row] = strain; sig_out[row] = sigma
        K_out[e] = K; F_out[e] = F
```

Dos observaciones sobre este esqueleto. Primera: la cinemática se recomputa dentro del bucle compilado; no hay caché de **B** (véase §5). Segunda: la física es exactamente la del camino por elemento, porque `_compute_kinematics` y `_compute_integrands` son las mismas funciones `@njit` que ya usa `Quad4.compute_element_state`; el kernel por lotes es el bucle exterior compilado, no una reimplementación.

Para los sólidos de orden superior 2D y 3D, que hoy comparten una base (`_HigherOrderSolid2D`, `_HigherOrderSolid3D`) parametrizada por las funciones de forma, el kernel por lotes se construye

una vez por base y recibe las funciones de forma compiladas como argumento, de modo que Quad8, Quad9, Tri6, Hex20, Hex27 y Tet10 lo heredan sin código propio.

4.3 Contrato opcional del material

El material declara el esquema de su estado y un kernel puntual con firma uniforme:

```
class VonMises2D(Material):
    STATE_SCHEMA = {"eps_p": (4,), "alpha": ()} # 5 columnas en S
    BATCH_KERNEL = _j2_point_kernel

@njit(cache=True)
def _j2_point_kernel(strain, S_row, p):
    # p: tupla de parametros (sigma_y, H, K, G, C_e, tol_rel, tol_abs, ...)
    eps_p_old = S_row[0:4]; alpha_old = S_row[4]
    sigma, C, eps_p_new, alpha_new = _compute_j2_plane_strain(
        strain, eps_p_old, alpha_old, p[0], p[1], p[2], p[3], p[4], p[5])
    S_new = np.empty(5); S_new[0:4] = eps_p_new; S_new[4] = alpha_new
    return sigma, C, S_new
```

El adaptador tiene una decena de líneas y llama al return mapping existente: no duplica física. La tolerancia de admisibilidad, que hoy se calcula en Python en cada llamada (`admissibility_tol`, 129 ms en el perfil), pasa a evaluarse dentro del kernel con las mismas constantes del ADR 0006, como ya hace el Newton local de plane stress desde la auditoría.

`STATE_SCHEMA` tiene valor incluso sin kernel por lotes: basta para que el estado se almacene como arreglos y el commit deje de copiar diccionarios. Por eso se propone que sea obligatorio en todo material nuevo (cuesta una línea) y que `BATCH_KERNEL` sea opcional.

4.4 Estado interno por arreglos

Cada familia mantiene un `FamilyState` con cuatro arreglos: estado committed y trial, (P, n_{state}) con $P = N \cdot n_{gp}$, y esfuerzos committed y trial, (P, n_{σ}) . Las operaciones son:

- **evaluación:** el kernel lee `S_committed` y escribe `S_trial`;
- **commit:** intercambio de referencias entre trial y committed; coste nulo;
- **bisección o rechazo del paso:** nada que hacer; el siguiente ensamblaje sobrescribe el trial desde el committed, exactamente la semántica trial/commit actual de `ElementState`.

La compatibilidad hacia atrás se resuelve en `ElementState`. En una familia vectorizada, las listas `vars` y `stresses` del estado, indexadas por punto de Gauss, pasan a ser vistas que construyen el diccionario bajo demanda a partir de la fila correspondiente de `S` y del esquema; asignar en `vars_trial` escribe la fila. Así `compute_gauss_state`, el exportador VTK y los tests que leen el estado siguen funcionando sin cambios, y el coste del diccionario sólo se paga cuando alguien lo pide.

4.5 Cinemática

El kernel recibe siempre `X` de referencia y `u`, y recomputa jacobiano, su inversa y `B` en cada evaluación. Invertir un jacobiano de 3×3 son unas decenas de operaciones, despreciables frente al return mapping, y en código compilado no tienen el coste de despacho que hoy las encarece. Esta elección hace al diseño indiferente a la cinemática:

- **lineal** (todos los sólidos actuales): `B` depende sólo de `X`; se recomputa igualmente, por uniformidad;
- **corrotacional** (armaduras, cables y marco 2D corrotacionales): el kernel reconstruye la configuración corriente a partir de `X + u`, como hoy hace cada elemento en `_current_geometry`;

- **lagrangiana total** (futura): las derivadas respecto a $\mathbf{X}$ son fijas y el gradiente de deformación $\mathbf{F}$ se evalúa por punto; encaja sin cambio de estructura;
- **lagrangiana actualizada** (futura): las derivadas se toman respecto a $\mathbf{x} = \mathbf{X} + \mathbf{u}$ y se recomputan; el kernel ya recibe ambos.

La topología COO, que depende sólo de la conectividad, sigue siendo válida con grandes desplazamientos.

4.6 Ensamblador

`assemble_non_linear_system` recorre las familias. Para las vectorizadas, invoca el kernel por trozos (véase §5) y vuelca `K_out` directamente en `data[ptr:ptr+n]` de la COO cacheada, y `F_out` en `F_int` con los índices precalculados. Para las no vectorizadas, aplica el bucle por elemento actual. Después construye la CSR exactamente como hoy. `commit_all_states` conmuta los arreglos de cada familia y llama a `commit_state` en los elementos no vectorizados; `prepare_all_steps` no cambia.

Los solvers (Newton estático, arc-length, Newmark, HHT, θ -method, explícito) no se tocan: consumen `assemble_non_linear_system`, `commit_all_states` y `reduce`, cuyos contratos se mantienen. Tampoco cambian `build_solve_result`, el exportador ni `compute_gauss_state`.

5 Memoria

5.1 Lo que se ahorra

El estado por arreglos reduce la memoria del estado interno de 462 a 80 B por punto de Gauss (tabla 2), once veces menos, y elimina la duplicación transitoria del commit. En un modelo 3D de 100 000 Hex8 con J2 (800 000 puntos de Gauss y siete variables de estado) son 90 MB frente a los 740 MB actuales.

5.2 Lo que no se debe cachear

Precalcular $\mathbf{B}$ y $\det \mathbf{J}$ por punto de Gauss es la variante que se pagaría en memoria. La tabla 3 compara su coste con lo que ya ocupa cada elemento en la topología COO cacheada (una entrada de 8 bytes de datos más dos índices de 4 bytes por coeficiente de $\mathbf{K}_e$).

Elemento	Caché de $\mathbf{B}$ por elemento	Entradas COO por elemento
Quad4 (4 gp, 3×8)	0,8 KB	1,0 KB
Quad8 (9 gp, 3×16)	3,5 KB	4,1 KB
Hex8 (8 gp, 6×24)	9,2 KB	9,2 KB
Hex20 (27 gp, 6×60)	77,8 KB	57,6 KB
Hex27 (27 gp, 6×81)	105 KB	105 KB

Tabla 3: Caché de $\mathbf{B}$ frente a la memoria ya comprometida por elemento en la COO.

Cachear $\mathbf{B}$ duplica el consumo por elemento en todas las familias y en 3D es prohibitivo: 10 000 Hex20 supondrían 780 MB sólo de $\mathbf{B}$. La propuesta no la cachea; recomputarla en el kernel cuesta menos que el return mapping. Lo único cacheado es la geometría de referencia (N, n_{nodos}, d) : 64 B por Quad4, 192 B por Hex8.

5.3 Temporales acotados por trozos

Los arreglos temporales del lote (deformaciones, tangentes, matrices elementales) se acotan procesando la familia por trozos de n_c elementos. El tamaño de trozo se fija con un presupuesto de memoria $M_{\text{máx}}$ (por ejemplo 64 MB):

$$n_c = \left\lfloor \frac{M_{\text{máx}}}{8 [n_{\text{gp}} (n_{\sigma} n_{\text{dof}} + n_{\sigma}^2 + 2n_{\sigma}) + n_{\text{dof}}^2 + n_{\text{dof}}]} \right\rfloor, \quad (2)$$

donde el corchete cuenta los dobles por elemento de **B**, **C**, deformación y esfuerzo por punto, más **K_e** y **F_e**. Para Hex20 son unos 90 KB por elemento, así que un presupuesto de 64 MB procesa 700 elementos por trozo; el pico no crece con el tamaño de la malla. Las matrices elementales del trozo se vuelcan a **data** y el temporal se reutiliza.

5.4 El techo real

Con o sin vectorización, lo que limita el tamaño del modelo es la factorización dispersa: el relleno del LU en 3D multiplica varias veces los no nulos de **K**. La palanca ahí es otra y ya está en el despachador algebraico (ADR 0003): Cholesky vía scikit-sparse para sistemas simétricos definidos positivos, con aproximadamente la mitad de memoria que LU, y a más largo plazo un solver iterativo con preconditionador para 3D grande. Ninguna depende de cómo se ensamble.

6 Rendimiento esperado y cómo validarlo

Con el bucle exterior compilado, el coste por punto de Gauss queda en la aritmética: cinemática (unas 100 operaciones), return mapping J2 (unas 200) y producto **B^TCB** (unas 600 para Quad4). Son del orden de 0,3 a 1 μ s por punto, es decir, entre 2 y 4 μ s por Quad4 frente a los 120 a 170 actuales: entre 30 y 60 veces, con margen hasta 100 en elementos con más puntos de Gauss, donde la sobrecarga relativa de hoy es mayor. El commit pasa de 0,4 s a un intercambio de referencias.

Tras esa ganancia el ensamblaje deja de dominar: para 20 000 grados de libertad la factorización LU cuesta unos 50 ms y pasa a ser el término principal de cada iteración de Newton, lo que a su vez motiva el Cholesky del ADR 0003. La estimación debe validarse con un benchmark reproducible antes de aceptar el ADR: la misma malla de 100×100 Quad4 con J2 y una de $20 \times 20 \times 20$ Hex8, midiendo ensamblaje, commit y solve por separado. El script de medición usado para las tablas de este documento se incluye en el apéndice.

7 Verificación

1. **Equivalencia entre caminos.** Para cada combinación registrada elemento $\times$ material compatible, sobre una malla pequeña con geometría distorsionada y un campo de desplazamientos que plastifique o dañe, comparar **K**, **F_{int}**, esfuerzos y estado trial entre el camino por elemento y el camino por lotes. Tolerancia: relativa 10^{-14} (ambos caminos ejecutan las mismas funciones compiladas; la única diferencia admisible es el orden de suma). La comprobación se añade a `test_element_contract_sweep.py` y `test_material_contract_sweep.py`, que ya iteran los registros, así que una combinación nueva entra sin que nadie la añada a mano.
2. **Semántica trial/commit.** Reproducir con familias vectorizadas los tests existentes de bisección (el trial de un paso rechazado no contamina el committed), de `prepare_step` y de `internal_forces` tras el commit.

3. **Vistas de estado.** `state.vars[idx]` y `state.stresses[idx]` devuelven en una familia vectorizada lo mismo que en una no vectorizada; el exportador VTK y `compute_gauss_state` producen salidas idénticas.
4. **Benchmarks de validación.** Lamé, NAFEMS LE1 y LE10, Hill J2, Cook plane strain y los cruces 2D–3D deben dar cifras idénticas: son el criterio de que la física no se movió.
5. **Rendimiento.** Umbral de aceptación del benchmark: al menos 20 veces en el ensamblaje de Quad4 con J2 y al menos 10 veces en Hex8, sin superar el presupuesto de memoria de trozo.

8 Plan por fases

Cada fase deja la suite verde y es útil por sí sola; ninguna obliga a la siguiente.

Fase 0 — Benchmark reproducible.

Script en `examples/benchmarks/` que mide ensamblaje, commit y solve en las dos mallas de referencia y guarda las cifras de partida. Criterio: cifras publicadas en `STATUS.md`.

Fase 1 — Estado por arreglos.

`STATE_SCHEMA` en los materiales con historia, `FamilyState` y las vistas en `ElementState`; commit sin copia profunda. Sin kernel por lotes todavía: el camino por elemento sigue escribiendo el estado, pero en las filas de los arreglos. Criterio: suite verde, memoria del estado dividida por diez, commit despreciable.

Fase 2 — Kernel por lotes para sólidos 2D lineales.

`BATCH_KERNEL` en Quad4 y Tri3 y en la base de orden superior 2D; adaptadores puntuales para Elastic2D, Orthotropic2D, VonMises2D, DruckerPrager2D e IsotropicDamage2D. Criterio: equivalencia 10^{-14} en el barrido y umbral de rendimiento en Quad4.

Fase 3 — Sólidos 3D.

Hex8, Tet4 y la base de orden superior 3D con los materiales 3D. Criterio: equivalencia y umbral en Hex8; presupuesto de trozo respetado con Hex27.

Fase 4 — Elementos estructurales 1D.

Armaduras, cables y marcos, incluidos los corotacionales, con los materiales 1D. Aquí la ganancia es menor (pocos puntos de Gauss) y la fase puede diferirse si el benchmark no la justifica.

Fase 5 — Térmicos.

Quad4Thermal y Hex8Thermal con `ThermalConduction`; misma estructura con `FLUX_DIM` en lugar de `STRAIN_DIM`.

Fuera de alcance de todas las fases: `CST_Embedded2D` y los materiales cohesivos. Su estado (`DiscontinuityState`, Newton local del salto) no encaja en arreglos regulares y su número de elementos es pequeño; permanecen en el camino por elemento sin penalización de diseño.

8.1 Coste del componente $N + 1$

Componente nuevo	Hoy	Con la propuesta
Material sin historia	<code>compute_state</code>	igual; <code>STATE_SCHEMA = {}</code> ; kernel puntual opcional (5 líneas)
Material con historia	<code>compute_state</code> + kernel <code>@njit</code>	igual + <code>STATE_SCHEMA</code> (1 línea, obligatoria); adaptador puntual opcional (10 líneas)
Sólido sobre una base existente	funciones de forma + <code>FACE_NODES</code>	igual; hereda el kernel por lotes de la base
Elemento con cinemática nueva	<code>compute_element_state</code>	igual; kernel por lotes opcional, escrito con las mismas funciones <code>@njit</code> del camino por elemento

Tabla 4: Lo que debe escribir quien añade un componente, antes y después.

9 Alternativas consideradas y descartadas

Cachear B por punto de Gauss.

Duplica la memoria por elemento y es prohibitiva en 3D (tabla 3); además introduce una hipótesis de geometría fija que las formulaciones futuras no cumplen.

Reescribir elementos y materiales en C++ con pybind11.

Rompe el modelo de colaboración del proyecto (Reglas.md §2): el mantenedor es una IA que trabaja en Python con contratos declarativos, y una segunda base de código compilada duplica cada componente. Numba ofrece la compilación sin abandonar el lenguaje.

Clases `jitclass` de Numba para elementos.

Sustituyen los contratos declarativos por tipos estáticos, impiden el fallback por elemento y complican el registro y el descubrimiento automático.

Paralelizar el bucle por elemento con procesos.

La sobrecarga de despacho se multiplica, no se elimina; la serialización de elementos y estado cuesta más que lo que se ahorra. El paralelismo tiene sentido después de vectorizar, sobre los trozos, con `prange` de Numba.

Optimizar la construcción de la matriz dispersa.

No es el cuello de botella: 10 ms de 1 660.

10 Decisiones abiertas para el ADR

1. Si `STATE_SCHEMA` es obligatorio en todo material (propuesta: sí, con `{}` para los materiales sin historia).
2. Tolerancia de equivalencia entre caminos (propuesta: relativa 10^{-14} ; si algún reordenamiento de sumas la excede, documentar el caso y fijar 10^{-12} para ese material).
3. Ubicación del código: `solidum/math/batch/` para `FamilyState`, construcción de familias y volcado; los kernels por lotes junto a cada elemento, como hoy los `@njit`.
4. Presupuesto de memoria por trozo y si se expone como parámetro del `Assembler` (propuesta: constante en `constants.py`, 64 MB, con override por argumento).

5. Si la fase 4 (elementos 1D) se ejecuta o se difiere según el benchmark.

Lo que este documento no autoriza

Nada de lo anterior se implementa hasta que el ADR se apruebe. La vectorización es un cambio arquitectural grande según Reglas.md §4, y las fases 1 y 2 modifican cómo se almacena el estado interno, que es zona gris (semántica trial/commit) y requiere validación conceptual del usuario.

A Reproducción de las mediciones

Las tablas 1 y 2 se obtuvieron con el siguiente script (Python 3.14, NumPy 2.3, SciPy 1.17, Numba con cache=True), sobre el estado del repositorio en el commit 90eff55.

```
import cProfile, pstats, tracemalloc, numpy as np, solidum
from solidum import Domain, Quad4, VonMises2D, Assembler

def mesh(nx, ny, mat):
    d = Domain(); nid = 1
    for j in range(ny + 1):
        for i in range(nx + 1):
            d.add_node(nid, [i / nx, j / ny]); nid += 1
    eid = 1
    for j in range(ny):
        for i in range(nx):
            n0 = 1 + i + j * (nx + 1)
            nodes = [d.nodes[n0], d.nodes[n0 + 1],
                    d.nodes[n0 + nx + 2], d.nodes[n0 + nx + 1]]
            d.add_element(Quad4(eid, nodes, mat)); eid += 1
    d.generate_equation_numbers()
    return d

d = mesh(100, 100, VonMises2D(E=1e3, nu=0.3, sigma_y=1e9))
asm = Assembler(d)
U = np.zeros(d.total_dofs)
asm.assemble_non_linear_system(U)           # calienta el JIT y la topologia

pr = cProfile.Profile(); pr.enable()
asm.assemble_non_linear_system(U)
pr.disable()
pstats.Stats(pr).sort_stats("tottime").print_stats(8)

tracemalloc.start()
s0 = tracemalloc.take_snapshot()
asm.assemble_non_linear_system(U)
s1 = tracemalloc.take_snapshot()
asm.commit_all_states()
s2 = tracemalloc.take_snapshot()
mb = lambda a, b: sum(x.size_diff for x in b.compare_to(a, "lineno")) / 2**20
print("ensamblaje:", mb(s0, s1), "MB; commit:", mb(s1, s2), "MB")
```